Travail noté 3 - INF 1220

Nom : *Lépine*
Prénom : *Alexandra*
Code permanent : *LEPA18549607*
Date de remise : 4 avril 2026

Question #1

Analyse préliminaire

Je dois écrire un programme Java qui calcul la somme des enombres de 1 jusqu'à 10 000, en incluant 1 et 10 000. Toutefois, je dois omettre les nombres qui sont divisible par 3 et je dois omettre les nombres dont le chiffre de la centaine est 2 ou 3. Puis, je dois expliquer ma solution.

Par la suite, je dois justifier tous les qualifiant utilisée. Si j'utilise le mot-clé "static" je dois le justifier.

À la fin, je dois répondre à la question : "Est-ce que la somme peut être représentée par une variable de type INT ?"

Code source

Lien du code source

```
public class TN3_Q1 {
    public static void main (String [] args) {
        long somme = 0;
        for (int i = 1; i <= 10000; i++) {
            // Condition 1 : ignorer les nombres divisible par 3
            if (i % 3 == 0) {
                continue;
            }
            int centaine = (i / 100) % 10;
            // condition 2 : ignorer les nombres dont les chiffres de la
centaine est 2 ou 3
            if (centaine == 2 || centaine == 3) {
                continue;
            }
            somme += i;
        }
        System.out.println("La somme est " + somme);
    }
}
```

Sortie du programme

```
La somme est 26937467
```

Description du programme

J'ai utilisé le mot-clé public, car ceci rend la classe et la méthode accessible à l'extérieur du paquet. Java a besoin que main soit public pour effectuer le démarrage.

J'ai utilisé le mot-clé Static, car ceci signifie que la méthode appartient à la classe elle-même et non à un objet, ceci est indispensable pour main.

J'ai utilisé void, car la méthode ne retourne à aucune valeur. Le résultat de la somme est affiché dans la console et non renvoyé à une source externe.

J'ai utilisé le type long, car la somme calculée dépasse la capacité du type int, qui est entre -2^{31} et 2^{31-1}, long a une capacité entre -2^{63} et 2^{63-1}, donc par sécurité advenant plusieurs additions j'ai choisi de débuté par long, car long peut contenir des nombres beaucoup plus grands.

Pour la condition en lien avec l'omission du chiffre de la centaine 2 ou 3, j'ai utilisé le type int, car celle-ci représente un chiffre entre 0 et 9. Comme il s'agit d'un entier sans décimale et de petite taille, le type int à la capacité d'effectuer la condition. De plus, les opérations arithmétiques / et % retournent des valeurs de type entier lorsqu'elles sont appliquées a des entiers.

Pour revenir à la question "Est-ce que la somme peut être représentée par une variable de type int ?" La réponse à cette question est oui, mais ce n'est pas recommandé. La somme calculée est 26937267. Le type int peut contenir jusqu'à 2^{31-1}, techniquement cette valeur convient au type int. Par contre, les sommes intermédiaires pendant le calcul pourraient dépasser cette limite si nous effectuons des modifications des conditions. Pour respecter les bonnes pratiques et éviter des problèmes, on utilise long qui a une valeur allant jusqu'à 2^{63-1}.

Question #2

Analyse préliminaire

Je dois écrire un programme Java qui affiche à l'écran toutes les chaînes de 4 caractères en respectant les critères suivants :

- Chaînes de 4 caractères en ordre lexicographique (ordre du dictionnaire)
- Doit afficher une chaîne de 4 caractères par ligne
- Seuls les caractères suivants sont permis : a,b,c,d
- Il est possible de réutiliser plus d'une fois un même caractère dans une chaîne
- Le caractère "b" doit toujours être immédiatement suivi du caractère "a"
- Une même chaîne ne peut comporter à la fois le caractère "d" et le caractère "a"
- Une fois terminer le programme affiche a l'écran le nombre de chaînes affichées
- Le programme ne reçoit aucune entrée
- Le code ne doit pas comprendre les chaînes de caractères pré-calculés : il doit les générer lors de l'exécution
- Le programme ne doit pas contenir plus de 200 courtes lignes de code

- Le résultat du programme doit être le même à chaque exécution

Code source

Lien du code source

```
public class TN3_Q2{
     public static void main(String[] args) {
        int count = 0;
        //Générer toutes les chaînes de 4 caractères
       char [] letters = {'a', 'b', 'c', 'd'};
        char [] s = new char [4];

        for (char c1 : letters) {
            s[0] = c1;
            for (char c2 : letters) {
                s[1] = c2;
                for (char c3 : letters) {
                    s[2] = c3;
                    for (char c4 : letters) {
                        s[3] = c4;

                        if (isValid(s)) {
                            System.out.println (new String (s));
                            count++;
                        }
                    }
                }
            }
        }
    System.out.println(count);
    }

private static boolean isValid(char[] s) {
    // Condition : une chaîne ne peut comporter le caractère "d" et le
caractère "a"
    boolean hasA = false;
    boolean hasD = false;

    for (int i = 0; i< s.length; i++) {
        if (s[i] == 'a') hasA = true;
        if (s[i] == 'd') hasD = true;
        // Condition : le caractère "b" doit être suivi immédiatement du
caractère "a"
        if (i<s.length - 1 && s[i] == 'b' && s[i + 1] != 'a'){
            return false;
        }
    }
            return ! (hasA && hasD);
    }
```

```
}
```

Sortie du programme

```
aaaa
aaab
aaac
aaba
aaca
aacb
aacc
abaa
abab
abac
acaa
acab
acac
acba
acca
accb
accc
baaa
baab
baac
baba
baca
bacb
bacc
caaa
caab
caac
caba
caca
cacb
cacc
cbaa
cbab
cbac
ccaa
ccab
ccac
ccba
ccca
cccb
cccc
cccd
ccdb
ccdc
ccdd
cdcb
```

```
cdcc
cdcd
cddb
cddc
cddd
dccb
dccc
dccd
dcdb
dcdc
dcdd
ddcb
ddcc
ddcd
dddb
dddc
dddd
63
```

Description du programme

J'ai utilisé le mot-clé public, car ceci rend la classe et la méthode accessible à l'extérieur du paquet. Java a besoin que main soit public pour effectuer le démarrage.

J'ai utilisé le mot-clé Static, car ceci signifie que la méthode appartient à la classe elle-même et non à un objet. Main doit petre static car la JVM l'appelle sans créer d'instance de la classe. De plus, isValid doit être static car elle est appelée dans main sans créer d'objets.

J'ai utilisé int pour count, car le type int contient des données en Java qui stocke des nombres entiers et sans décimales. Il permet de compter les chaînes, les chaînes contiennent seulement des entiers.

J'ai utilisé le mot-clé private, car celui-ci limite l'accès à la méthode à l'intérieur de la classe unique. Elle n'a pas besoin d'être accessible depuis les autres classes.

J'ai utilisé void, car la méthode ne retourne à aucune valeur. Le résultat de la somme est affiché dans la console et non renvoyé à une source externe.

J'ai utilisé boolean, car la méthode retourne à true or false, ce qui était essentiel pour l'application de la condition "une chaîne ne peut comporter à la fois le caractère 'd' et le caractère 'a'.

J'ai utilisé l'opérateur de comparaison == (Égal à) pour comparer si b = a dans la chaîne de caractères. L'opérateur retourne également a true or false.

J'ai utilisé l'opérateur logique && (Et logique), qui signifie "et" en logique booléenne. L'opérateur logique permet de combiner plusieurs combinaisons et retourne à true si et seulement si toutes les conditions sont vraies.

Question-boni

Analyse préliminaire

Je dois écrire une classe Java qui génère automatiquement un tableau de Sudoku. La grille de Sudoku doit respecter les critères suivants :

- Possible d'utiliser la méthode Random.nextInt
- La grille doit avoir le type int [] []
- Le tableau doit contenir 9 rangées et 9 colonnes
- Chaque colonne doit contenir les chiffres de 1 à 9, sans répétition
- Chaque rangée doit contenir les chiffres de 1 à 9, sans répétition
- Il n'est pas requis que le code soit rapide mais doit être complété dans un délai raisonnable, max 2 minutes

Code source

Lien vers le code de la grille Sudoku

```
import java.util.Random;
public class TN3_QB{
    private int[][] grille = new int [9][9];
    private Random random = new Random ();

    public int [][] generer (){
        if (remplir (0,0)){
            return grille;
        }else{
            System.out.println("Impossible de générer la grille");
            return null;
        }
     }
     // Application des contraintes : Lignes, colonnes, blocs 3x3
     private boolean remplir (int ligne, int col){
        if (ligne ==9){
            return true;
        }
        // Calculer la prochaine case
        int nextLigne = (col == 8) ? ligne + 1 : ligne;
        int nextCol = (col + 1) % 9;
        // Création d'un tableau avec des valeurs de 1 à 9 en ordre
aléatoire
        int[] chiffres = ordreAleatoire ();

        for (int nbEssai : chiffres) {
            if (isValid (ligne, col, nbEssai)){
                grille [ligne][col] = nbEssai;
                // Récursivité de la prochaine case
                if (remplir(nextLigne, nextCol)){
                    return true;
                }
                // Réinitisalition, nbEssai ne mèene pas à une solution
                grille[ligne][col] =0;
            }
        }
```

```
            return false;
        }
        // Génération de l'ordre aléatoire avec nextInt(bound)
        private int [] ordreAleatoire(){
            int[] tab = {1,2,3,4,5,6,7,8,9};

            for (int i =0; i < 9; i++){
                int j = random.nextInt(9);
                int temp = tab [i];
                tab[i] = tab[j];
                tab[j] = temp;
            }
            return tab;
        }
        //Vérifier que les conditions suivantes soient respectés :
        //Un chiffre n'apparait pas à la même ligne
        //Un chiffre n'apparait pas à la même colonne
        //Un chiffre n'apparait pas dans le même bloc 3x3

        private boolean isValid(int ligne, int col, int valeurs){
            // vérification de la ligne
            for (int k = 0; k < 9; k++){
                if (grille[ligne][k] == valeurs) return false;
            }
            // véfication de la colonne
            for (int k = 0; k < 9; k++) {
                if (grille[k][col]== valeurs) return false;
            }
            // Vérification du bloc 3x3
            int blocLigne = (ligne / 3) * 3;
            int blocCol = (col / 3) * 3;

            for (int i = blocLigne; i < blocLigne + 3; i++){
                for (int j = blocCol; j < blocCol + 3; j++){
                    if (grille[i][j] == valeurs) return false;
                }
            }
            return true;
        }
        // Affichage de la grille dans la console
        private void afficher (){
            System.out.println("Grille de Sudoku Générée :");
            for (int i = 0; i < 9; i++) {
                if (i % 3 == 0 && i != 0){
                    //Afficher la séparation de la grille
                    System.out.println("------+-------+--------");
                }
                for (int j = 0; j < 9; j++){
                    if (j % 3 == 0 && j != 0){
                        System.out.print(" | ");
                    }
                    System.out.print(grille [i][j] + " ");
                }
                System.out.println();
```

```
        }
    }
    // Tester la grille
    public static void main(String[] args) {
        long debut = System.currentTimeMillis();

        TN3_QB generateur = new TN3_QB();
        int[][] Sudoku = generateur.generer();

        if (Sudoku != null){
           generateur.afficher();
           long temps = System.currentTimeMillis() - debut;
           System.out.println("\n Temps de génération :" + temps + "ms");
        }
    }
   }
```

Sortie du programme

Essai 1 :

```
Grille de Sudoku Générée :
1 6 4  | 2 9 8  | 3 5 7
5 3 8  | 1 7 6  | 9 4 2
9 7 2  | 3 5 4  | 1 8 6
------+-------+--------
2 9 6  | 7 8 1  | 5 3 4
4 8 7  | 9 3 5  | 6 2 1
3 1 5  | 6 4 2  | 8 7 9
------+-------+--------
6 4 9  | 8 2 3  | 7 1 5
8 2 1  | 5 6 7  | 4 9 3
7 5 3  | 4 1 9  | 2 6 8

 Temps de génération :1ms
```

Essai 2 :

```
Grille de Sudoku Générée :
9 8 2  | 7 4 1  | 6 5 3
3 7 4  | 2 5 6  | 8 9 1
6 5 1  | 3 8 9  | 4 7 2
------+-------+--------
8 2 3  | 1 9 7  | 5 4 6
7 4 6  | 5 2 3  | 1 8 9
5 1 9  | 8 6 4  | 2 3 7
------+-------+--------
2 9 8  | 6 7 5  | 3 1 4
```

```
 4 3 5  | 9 1 2  | 7 6 8
 1 6 7  | 4 3 8  | 9 2 5

  Temps de génération :3ms
```

Description du programme

J'ai utilisé private int[][] afin de créer la structure principale de mon code et ceci me permet de créer un tableau à deux dimensions de 9 x 9 cases, où chaque case stocker un entier.

Private Random random = new Random me permet de générer des nombres aléatoires pour générer à chaque fois une nouvelle grille de Sudoku.

La fonction générale est public int[][] generer () qui me permet de générer la grille, ceci est mon point d'entrée pour la création de ma grille de Sudoku.

J'ai utilisé private boolean remplir pour remplir la grille case par case et ceci permet au code d'arrêter lorsqu'il y un erreur ou lorsque la grille est pleine.

J'ai utilisé les boucles for if pour tester les chiffres possibles, valider si les conditions sont respectées et s'assurer de la récursivité.

La fonction private int [] ordreAleatoire() me permet d'attribuer un chiffre de 1 à 9 aléatoire dans la grille de Sudoku.

Pour valider les règles du Sudoku, j'ai utilisé private boolean isValid. Ceci me permet de valider si le chiffre n'est pas présent dans la ligne et dans la colonne et vérifie que les chiffres de 1 à 9 sont bien présents dans le bloc 3x3.

J'ai utilisé private void afficher() pour afficher la grille proprement avec l'ajout de lignes et de séparateur.

La fonction public static void main (String [] args) me permet de tester le programme.

Pour calculer le temps que prend le programme pour générer la grille de Sudoku, j'ai utilisé long debut = System.currentTime.Millis().